

PRELIMINARY ON-CHAIN FORENSIC REPORT

JaredFromSubway / @jaredsmev MEV Bot Incident

Ethereum Mainnet - June 2026

Prepared for: @jaredsmev / JaredFromSubway

Prepared by: @TheRoadTo1BTC - Independent on-chain security researcher

Date: June 21, 2026

Classification: Defensive security research / incident response support

Researcher note

This report is intended as a useful, good-faith technical contribution to the incident response process. It does not contain exploit code intended for reuse against third parties. The objective is to help identify the failure mode, preserve the evidence trail, reproduce the issue safely, and harden the affected automation against similar counter-MEV traps.

Contact / X	@TheRoadTo1BTC
BTC tip address	bc1qc7yc7xrmle9rdygp6xppn22uv26e68ymn4n39
Project site	https://roadto1btc.vercel.app
Public note	Can the internet help one random person reach 1 BTC? 100,000,000 sats. Public. On-chain. No token. No promises.
Gratuity note	If this work is useful and saves investigation time, voluntary sats are appreciated. No recovery claims, no token, no promises.

Bitcoin tip QR



Important limitation: the findings below are based on public sources, explorer-visible on-chain data, and public Blockaid statements. Final conclusions require private traces, bot logs, bundle data, and the exact executor/searcher code path.

Table of Contents

1. Executive Summary
2. Key Findings
3. Evidence and Confidence Levels
4. Known On-Chain Facts
5. Incident Timeline
6. Asset Flow Summary
7. Technical Attack Narrative
8. Root Cause Hypotheses
9. Indicators of Compromise
10. Foundry Mainnet-Fork Reproduction Plan
11. Approval Investigation Plan
12. Non-Draining Defensive PoC
13. Hardening Recommendations
14. Operational Response Plan
15. Suggested Deliverables for JaredFromSubway
16. Open Questions
17. References

1. Executive Summary

A publicly reported exploit affected the @jaredsmev / JaredFromSubway MEV infrastructure on Ethereum. The most visible transaction associated with the drain is `0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65`. Public explorer data and Blockaid reporting indicate that approximately USD 7.5M in liquid assets was removed from a JaredFromSubway MEV bot address through a transaction involving attacker-controlled contracts and open token approvals.

The most likely failure mode is not a private key compromise. The working hypothesis is an operational logic failure in the MEV pipeline: the bot or its executor interacted with contracts controlled by the attacker, granted or retained approvals over core assets, and the attacker later used those approvals to perform `transferFrom` calls against WETH, USDC, and USDT balances.

This class of incident should be treated as counter-MEV / adversarial searcher exploitation. It exploits assumptions made by automated trading infrastructure: if the searcher only optimizes for expected profit and does not enforce strict asset, spender, target, and post-state invariants, a malicious opportunity can be engineered to look profitable while leaving dangerous state behind.

Executive determination

Question	Preliminary determination	Confidence
Was there a public on-chain drain?	Yes. A transaction with many ERC-20 transfers from the tagged bot address is publicly visible.	High
Was the loss approximately USD 7.5M?	Public Blockaid statement and Etherscan-visible transfer values align with a ~USD 7.46M to ~USD 7.5M range.	High
Was this likely a private key compromise?	No direct evidence from public data. The token flow is more consistent with allowance/ <code>transferFrom</code> misuse.	Medium
Was the attacker using controlled contracts?	Blockaid explicitly describes attacker-controlled contracts. The transaction structure is consistent with that claim.	High
Can the exact vector be fully proven from public HTML alone?	No. Requires <code>debug_traceTransaction</code> , historical Approval events, and the bot executor code.	Medium

Recommended next step: identify the approval seed transaction(s). The sweep transaction is visible; the critical missing link is the earlier transaction or code path that allowed the sweeper or a related contract to spend WETH, USDC, and USDT from the bot address.

2. Key Findings

- A primary drain transaction is identified:
0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65.
- The transaction was reported by Etherscan as successful and associated with Ethereum mainnet block 25360696, with a timestamp of June 20, 2026 18:49:11 UTC in the prior public review.
- The public Blockaid post states that its exploit detection system detected an exploit involving @jaredsmev and that USD 7.5M was drained.
- The visible asset movement was concentrated in three highly liquid assets: WETH, USDC, and USDT.
- The affected source is identified in the public review as JaredFromSubway: MEV Bot 2 / 0x1f2F10D1C40777AE1Da742455c65828FF36Df387.
- The receiving address observed in the token transfers is 0x3e37f4A10d771Ba9dE44b6d301410b1BEdeA65d0.
- The most useful defensive deliverable is not a copy of the malicious .sol file. The useful deliverable is a reproducible forensic package: trace, approval history, non-draining PoC, guardrail patch, and monitoring rules.

3. Evidence and Confidence Levels

The following table separates confirmed observations from inferences. This is important because a strong forensic report should not overstate what can be proven from public sources alone.

Item	Evidence type	Confidence	Notes
Blockaid public alert	Public X post indexed by search result	High	States exploit involving @jaredsmev and USD 7.5M drained.
Primary transaction hash	Etherscan transaction result	High	Hash is public and can be independently rechecked.
Loss assets	Etherscan token transfer summary	High	WETH, USDC, USDT transfers visible.
Use of approvals / transferFrom	Blockaid secondary summary plus transfer pattern	Medium-High	Should be confirmed by debug_traceTransaction and historical Approval logs.
Attacker-controlled contracts	Blockaid summary	High	Exact contract graph still requires trace and creator analysis.
Private key compromise	No supporting public evidence	Low	Not the leading hypothesis based on current evidence.

4. Known On-Chain Facts

Primary transaction	0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65
Chain	Ethereum mainnet
Reported block	25360696
Reported timestamp	2026-06-20 18:49:11 UTC
External caller	0x5aF38735B215b00aa7C9f93fEd7ee415CeCB36e1
Contract called	0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52
Observed selector	0xc269a509
Victim/source	0x1f2F10D1C40777AE1Da742455c65828FF36Df387
Observed receiver	0x3e37f4A10d771Ba9dE44b6d301410b1BEdeA65d0

Affected tokens

Token	Contract	Approx. observed amount
WETH	0xC02aaA39b223FE8D0A0e5C4F27eAD9083	1,474.582523004994977792 WETH

	C756Cc2	
USDC	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48	2,870,573.12768 USDC
USDT	0xdAC17F958D2ee523a2206206994597C13D831ec7	2,035,760.155871 USDT

5. Incident Timeline

Time / phase	Event	Evidence / action
Pre-attack	Attacker likely deployed or prepared controlled contracts designed to be attractive to MEV automation.	Validate by tracing contract creators and deployment blocks.
Pre-attack	MEV system likely interacted with attacker-controlled contracts and created dangerous allowances or state.	Find Approval logs from the source address to suspected spenders.
2026-06-20 18:49:11 UTC	Primary sweep transaction executes successfully.	Transaction hash: 0x2be870...f3e65.
Post-sweep	WETH, USDC, and USDT moved from source bot address to receiver address.	ERC-20 transfer logs.
Post-sweep	Public alert published by Blockaid describing an exploit involving @jaredsmev and USD 7.5M drained.	Blockaid X post.
Current	Forensic priority is to identify the approval seed and harden the executor/searcher pipeline.	Run the procedures in Sections 10-13.

6. Asset Flow Summary

The drain appears to be a direct removal of liquid token balances from the bot-controlled address rather than a normal swap or arbitrage settlement. The meaningful distinction is that the receiving address is not a protocol pool receiving trade input; it is an attacker-controlled endpoint receiving assets from the bot source.

From	To	Asset	Approx. amount	Forensic meaning
0x1f2F...f387	0x3e37...65d0	WETH	1,474.582523 WETH	High-value native liquidity wrapper removed from bot balance.
0x1f2F...f387	0x3e37...65d0	USDC	2,870,573.12768 USDC	Stablecoin balance removed through token transfer.
0x1f2F...f387	0x3e37...65d0	USDT	2,035,760.155871 USDT	Stablecoin balance removed through token transfer.

Why this matters

A regular MEV execution path typically produces offsetting protocol interactions: swap inputs, swap outputs, pool transfers, callbacks, settlement, and final profit. A drain pattern has a different signature: core assets leave the executor/source account to a non-protocol receiver with no legitimate equivalent return. That is why post-state invariants must be enforced around balances and allowances.

7. Technical Attack Narrative

The working narrative is a counter-MEV trap. The attacker likely did not need to defeat cryptography. Instead, the attacker exploited a weakness in the decision-making layer of automated MEV execution. Searchers operate under time pressure, with complex simulations and routing logic. If the system treats arbitrary contracts as potential opportunity components without strict trust boundaries, those contracts can be weaponized.

Probable sequence

1. Attacker creates one or more contracts, tokens, or pools that appear to create an exploitable opportunity.

2. The JaredFromSubway searcher detects the opportunity and simulates a profitable path, or a path that passes its internal profit/risk checks.
3. The executor interacts with attacker-controlled contracts. During this interaction, it grants approvals, leaves approvals dirty, or routes calls through a spender controlled by the attacker.
4. The attacker later calls a sweeper contract. The sweeper uses the existing allowance to call transferFrom on WETH, USDC, and USDT.
5. The bot/source address loses liquid assets to the attacker receiver address.
6. The public chain records the token movements, but the exact reason the allowance existed must be reconstructed from prior transactions and debug traces.

Likely vulnerable design assumptions

- Contracts used in an opportunity were assumed to be economically adversarial but not state-adversarial.
- Profit simulation may not have treated residual approvals as a critical failure condition.
- The executor may have optimized for gas and latency by reusing unlimited or persistent approvals.
- The system may have lacked a deny-by-default policy for new spenders, new tokens, and new pool factories.
- The monitoring layer may not have paused execution when an unusual Approval or transferFrom risk appeared.

8. Root Cause Hypotheses

Hypothesis	Description	Evidence needed	Current confidence
H1: Open approval abuse	A spender contract had positive allowance for core assets and used transferFrom to sweep balances.	allowance at block 25360695; Approval logs; debug trace showing transferFrom.	High
H2: Malicious spender obtained approval during earlier trap trade	A fake opportunity caused the bot to approve a contract controlled by the attacker.	Transaction that emitted Approval from source to spender; source code/trace of opportunity.	Medium-High
H3: Arbitrary call target in executor	Executor allowed a strategy to call arbitrary external addresses without target allowlist.	Executor code path and call trace showing external call to attacker target.	Medium
H4: Callback or token-standard abuse	A malicious token/pool used callback or non-standard ERC-20 behavior to alter state.	Trace showing callback path, non-standard transfer/approve behavior, storage writes.	Medium
H5: Private key compromise	Attacker obtained signing keys and transferred assets.	Evidence of direct owner-signed transfers or off-chain compromise.	Low

Most likely cause root

The strongest current hypothesis is H1 plus H2: an adversarial opportunity created or exploited a spender allowance, then a later sweeper transaction used that allowance to call transferFrom against core tokens. This should be treated as a design and controls failure in the execution pipeline, not simply a one-line Solidity bug.

9. Indicators of Compromise

Type	Value	Recommended action
Transaction	0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65	Trace, decode calldata, extract logs.
Source / victim	0x1f2f10D1C40777AE1Da742455c65828FF36Df387	Review all approvals and token transfers.
External caller	0x5aF38735B215b00aa7C9f93fEd7ee415CeCB36e1	Cluster related activity and funding source.
Called contract	0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52	Inspect bytecode, creator, creation tx, traces.
Receiver	0x3e37f4A10d771Ba9dE44b6d301410b1BEdeA65d0	Track onward flows; share with monitoring partners.
Selector	0xc269a509	Decode against verified or decompiled ABI if available.
WETH	0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2	Check Approval and transferFrom logs.
USDC	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48	Check Approval and transferFrom logs.
USDT	0xdAC17F958D2ee523a2206206994597C13D831ec7	Check Approval and transferFrom logs.

10. Foundry Mainnet-Fork Reproduction Plan

The goal is to build a reproducible forensic package that anyone on the incident team can run. It should not rely on a malicious contract copy. It should replay the public transaction, extract the call graph, and validate whether allowances existed at the prior block.

Environment

```
mkdir jared-forensics
cd jared-forensics
forge init

export ETH_RPC_URL="https://YOUR_ARCHIVE_RPC"
export TX="0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65"
export BLOCK_BEFORE=25360695
```

Basic transaction inspection

```
cast tx $TX --rpc-url $ETH_RPC_URL
cast receipt $TX --rpc-url $ETH_RPC_URL
cast run $TX --rpc-url $ETH_RPC_URL
```

Full call trace

```
cast rpc debug_traceTransaction $TX '{"tracer":"callTracer","timeout":"120s"}' --rpc-url
$ETH_RPC_URL > trace.callTracer.json

cast rpc debug_traceTransaction $TX '{"tracer":"prestateTracer","timeout":"120s"}' --rpc-url
$ETH_RPC_URL > trace.prestate.json
```

Trace questions to answer

- Does 0xb84d... call WETH, USDC, and USDT directly?
- Do the token calls use transferFrom(address,address,uint256)?
- Is the first parameter of transferFrom equal to 0x1f2F...f387?
- Is the second parameter equal to 0x3e37...65d0 or an intermediate receiver?
- Are there any delegatecall, multicall, fallback, or callback paths?
- Which contract is the actual ERC-20 spender from the token contracts perspective?

11. Approval Investigation Plan

Finding the approval seed is the single highest-value forensic step. The sweep transaction proves the loss event; the approval seed explains how it became possible.

Block-before allowance checks

```
# WETH allowance from victim to suspected contract
cast call 0xc02aa39b223fE8D0A0e5C4F27eAD9083C756Cc2 "allowance(address,address)(uint256)"
0x1f2F10D1C40777AE1Da742455c65828FF36Df387 0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52 --block
$BLOCK_BEFORE --rpc-url $ETH_RPC_URL

# USDC allowance
cast call 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 "allowance(address,address)(uint256)"
0x1f2F10D1C40777AE1Da742455c65828FF36Df387 0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52 --block
$BLOCK_BEFORE --rpc-url $ETH_RPC_URL

# USDT allowance
```

```
cast call 0xdAC17F958D2ee523a2206206994597C13D831ec7 "allowance(address,address)(uint256)"
0x1f2F10D1C40777AE1Da742455c65828FF36Df387 0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52 --block
$BLOCK_BEFORE --rpc-url $ETH_RPC_URL
```

Historical Approval log search

Search WETH, USDC, and USDT logs for Approval events where owner equals 0x1f2F...f387 and spender equals either 0xb84d... or any address discovered in the call trace. If the actual spender differs from 0xb84d..., repeat the queries for each discovered intermediary.

```
# Approval(address indexed owner, address indexed spender, uint256 value)
# topic0 = keccak256("Approval(address,address,uint256)")
# owner topic = 32-byte left-padded victim address
# spender topic = 32-byte left-padded spender address
```

```
cast logs --from-block 0 --to-block 25360695 --address
0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2 'Approval(address indexed,address indexed,uint256)'
0x1f2F10D1C40777AE1Da742455c65828FF36Df387 0xb84db016324e8F2BFdD8DD9c260338AEE0A8DF52 --rpc-url
$ETH_RPC_URL
```

Expected outputs

- Approval seed transaction hash.
- Block number and timestamp of the approval seed.
- Exact spender address approved by the bot/source account.
- Approved value for each token.
- Whether approval was exact, large, or uint256 max.
- Whether the approval was later revoked, decreased, or left open.

12. Non-Draining Defensive PoC

The PoC should validate the control failure without reproducing the attacker drain against real assets. It should use a mock ERC-20, a mock executor, and a sweep simulator in local tests. The expected lesson is: if a spender has allowance, transferFrom works; if approval controls and post-state invariants are enforced, it fails.

PoC components

Component	Purpose	Safety property
MockToken	Local ERC-20 used only in tests.	No real assets.
VictimExecutorMock	Simulates a bot executor holding token balances.	Demonstrates risk of approvals.
SweepSimulator	Attempts transferFrom using allowance.	Non-malicious test artifact.
ApprovalGuard	Blocks approvals to non-allowlisted spenders and enforces max allowance.	Defensive patch.
Invariant tests	Ensure no dirty allowance remains after execution.	Regression control.

Critical invariant

For every core token T and for every non-allowlisted spender S:

```
allowance(address(executor), S) == 0
```

After every operation:

```
allowance(address(executor), approvedSpender) == 0
```

```
balanceDelta(coreAssets) >= -maxPermittedLoss
```

```
no external call target outside allowedTargets
```

Minimal defensive interface

```
interface IERC20Like {
    function approve(address spender, uint256 amount) external returns (bool);
    function allowance(address owner, address spender) external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
}
```

ApprovalGuard patch concept

```
contract ApprovalGuard {
    address public owner;
    mapping(address => bool) public allowedTokens;
    mapping(address => bool) public allowedSpenders;
    mapping(address => mapping(address => uint256)) public maxAllowance;

    error NotOwner();
    error TokenNotAllowed();
    error SpenderNotAllowed();
    error AllowanceTooHigh();
    error ApprovalFailed();

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }
}
```

```

constructor() { owner = msg.sender; }

function safeApprove(address token, address spender, uint256 amount) external onlyOwner {
    if (!allowedTokens[token]) revert TokenNotAllowed();
    if (!allowedSpenders[spender]) revert SpenderNotAllowed();
    if (amount > maxAllowance[token][spender]) revert AllowanceTooHigh();

    bool resetOk = IERC20Like(token).approve(spender, 0);
    if (!resetOk) revert ApprovalFailed();

    bool approveOk = IERC20Like(token).approve(spender, amount);
    if (!approveOk) revert ApprovalFailed();
}

function clearApproval(address token, address spender) external onlyOwner {
    bool ok = IERC20Like(token).approve(spender, 0);
    if (!ok) revert ApprovalFailed();
}
}

```

Non-draining test cases

7. Without approval, SweepSimulator.transferFrom must revert.
8. With open approval, SweepSimulator.transferFrom succeeds in local test, proving the risk.
9. With ApprovalGuard and untrusted spender, safeApprove must revert.
10. With allowlisted spender but excessive amount, safeApprove must revert.
11. With exact approved amount, operation may execute but must clear approval to zero afterwards.
12. Invariant test fails if any core token has a nonzero allowance to a non-allowlisted spender.

13. Hardening Recommendations

13.1 Deny-by-default trust model

Every address used by the searcher should be classified. Unknown address equals hostile address. New contracts, unverified contracts, proxies, recently deployed tokens, fake pools, and non-standard ERC-20s must be denied unless specifically allowed by policy.

13.2 Approvals

- Do not use unlimited approvals for core assets in the executor.
- Use exact-amount approvals only when absolutely necessary.
- Reset allowance to zero after each operation.
- Fail the operation if post-state allowance is nonzero for any non-allowlisted spender.
- Maintain per-token and per-spender max allowance values.
- Continuously monitor Approval logs where owner equals the bot/executor address.

13.3 Call-target controls

- Require target allowlisting for all external calls from the executor.
- Separate routers, factories, pools, token contracts, and arbitrary targets into different allowlist classes.
- Block delegatecall unless the implementation is pinned and audited.
- Reject fallback-only targets unless explicitly approved.
- Reject newly deployed contracts for a cool-down window unless manually approved.

13.4 Simulation controls

Pre-trade simulation should not only test profit. It should test adverse state changes and execution side effects.

Check	Fail condition
Balance delta	Core asset loss exceeds configured threshold.
Allowance delta	Any new or increased allowance to untrusted spender.
External calls	Target outside allowlist.
Token behavior	Transfer/transferFrom/approve returns abnormal data or mutates unexpected state.
Storage writes	Unexpected storage write in executor or critical dependent contract.
Delegatecall	Any delegatecall outside pinned implementation path.
Contract age	Target contract younger than policy threshold.

13.5 Loss limits

- Maximum loss per opportunity by asset.
- Maximum daily exposure by asset.
- Maximum per-spender exposure.
- Circuit breaker on first abnormal Approval event.
- Circuit breaker on direct transferFrom from bot address to non-protocol receiver.

13.6 Monitoring rules

```

ALERT if Approval.owner == BOT_ADDRESS and Approval.spender not in ALLOWED_SPENDERS
ALERT if Transfer.from == BOT_ADDRESS and token in CORE_ASSETS and Transfer.to not in
ALLOWED_RECEIVERS
ALERT if allowance(BOT_ADDRESS, spender) > maxAllowance[token][spender]
PAUSE if cumulative core-asset outflow in block > configured threshold
PAUSE if opportunity path includes unverified or newly deployed contract

```

14. Operational Response Plan

Immediate containment

13. Pause all searchers, executors, workers, and keepers associated with the affected source account.
14. Revoke approvals for WETH, USDC, USDT, and other core assets from the source and related executors.
15. Move unaffected funds to newly deployed, hardened executors or cold storage.
16. Rotate keys for deployers, operators, signing services, RPC keys, bundle relays, and servers.
17. Export logs: bundle submissions, simulations, opportunities, calldata, traces, RPC responses, and signer activity.
18. Tag involved addresses across internal monitoring and external platforms.

First 24 hours

19. Find approval seed transaction(s).
20. Build contract graph: creators, funders, called targets, receivers, and onward flows.
21. Review all other executors for similar allowances.
22. Deploy emergency watcher for Approval and Transfer events.
23. Publish or internally circulate a concise IOC advisory.

Seven-day hardening sprint

24. Implement ApprovalGuard or equivalent controls in executor code.
25. Add post-state invariant checks to the simulation engine.
26. Add target/spender/token allowlists with configuration review.
27. Add max-loss and max-exposure limits.
28. Add regression tests using the public attack transaction and the non-draining PoC.
29. Have an external reviewer inspect the patched code path.

15. Suggested Deliverables for JaredFromSubway

A useful incident support package should include concrete files and scripts, not only narrative analysis. The following package would be valuable to the affected operator.

Deliverable	Purpose	Priority
Mainnet-fork replay repo	Reproduce the drain transaction and inspect state at block-before.	P0
Trace decoder	Decode call graph, transferFrom calls, and selector 0xc269a509.	P0
Approval seed finder	Search Approval logs for source/spender/token combinations.	P0
IOC list	Share addresses, tokens, tx hashes, and selectors with monitoring partners.	P0
Non-draining PoC	Demonstrate the approval failure mode safely.	P1
ApprovalGuard patch	Prevent repeat exposure.	P1
Simulation invariant tests	Fail dangerous opportunities before bundle submission.	P1
Monitoring rules	Pause on abnormal approvals/outflows.	P1

Researcher contribution note

The author of this report is available to help convert the preliminary analysis into a runnable forensic repository, including call traces, event queries, approval seed identification, a non-draining Foundry PoC, and a defensive patch review. If this report or the follow-up package saves investigation time or helps prevent a repeat loss, voluntary sats are appreciated.

X / Twitter	@TheRoadTo1BTC
BTC tip	bc1qc7yc7xrmle9rdygp6xppn222uv26e68ymn4n39
Project site	https://roadto1btc.vercel.app
Project note	Can the internet help one random person reach 1 BTC? 100,000,000 sats. Public. On-chain. No token. No promises.

BTC tip QR



Project site QR



16. Open Questions

- What exact transaction first created or increased the relevant WETH/USDC/USDT approvals?
- Was the spender exactly 0xb84d... or an intermediate contract discovered only in trace?
- Was selector 0xc269a509 a custom batch sweep method, an obfuscated function, or a known ABI method?
- Were attacker-controlled contracts verified, proxied, recently deployed, or self-destructable?

- Did the bot simulation model allowance deltas and post-state invariants?
- Were the approvals unlimited, exact, or bounded?
- Did any previous failed or successful opportunities interact with the same contract cluster?
- Did the bundle use a private relay, builder payment, or explicit coinbase/builder incentive?
- Are there other JaredFromSubway executors with similar residual approvals?
- Were any non-core tokens or smaller assets also affected outside the primary transfer summary?

17. References

1. Blockaid public X post: https://x.com/blockaid_/status/2068433798757577198
2. JaredFromSubway / @jaredsmev public X post: <https://x.com/jaredsmev/status/2068581403265249714>
3. Etherscan transaction:
<https://etherscan.io/tx/0x2be8704f5a59b69e0b71f64aefdb99eb0e8ae9fb3926147c581910d71bcf3e65>
4. WETH token contract: <https://etherscan.io/token/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2>
5. USDC token contract: <https://etherscan.io/token/0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48>
6. USDT token contract: <https://etherscan.io/token/0xdAC17F958D2ee523a2206206994597C13D831ec7>